

HARDWARE DESIGN WITH VHDL

- ✓ Hardware Description Language
- ✓ Hardware description in VHDL
- ✓ Structure of a VHDL Model
- ✓ VHDL Basics
- ✓ Example: Finite State Machine

Javier Resano, *GAZ: Computer Architecture Group*



Departamento de
Informática e Ingeniería
de Sistemas

Universidad Zaragoza

Hardware Description Language

- What is HDL?
 - Languages created for circuit design:
 - » Gate level.
 - » Behavioral level.
 - The language structure suggests the hardware design.
- Why use HDL?
 - The design is much faster.
 - It allows to simulate before physical implementation.
 - The design tools take care of the automatable steps.

Describing hardware is not the same as programming software

Hardware Description Language

- VHDL
 - VHSIC (U.S. Government 1980).
 - IEEE VHDL'87.
 - www.vhdl.org
- Verilog
 - Developed by CADENCE.
 - IEEE 1364.
 - www.eda.org
- From C:
 - Handel C
 - System C
 - C/C++ (high-level synthesis)
 - High-level integration tools (e.g. Verilator)

Hardware Description Language

- VHDL
 - Description of the circuit structure
 - Decomposition into sub-circuits
 - Interconnection of sub-circuits
 - Behavior
 - Structure
 - Allows the specification of circuit functionality using a syntax akin to programming languages.
 - Allows simulation of the circuit prior to manufacturing
 - Test and compare alternatives without the need for hardware prototypes

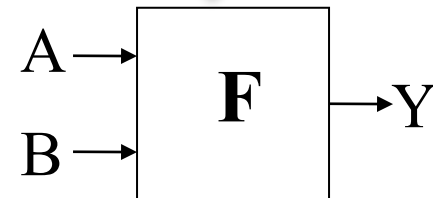
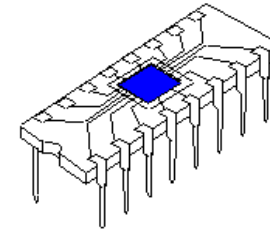
Hardware Description Languages

- Why VHDL?
 - One of the **most widely used** languages for IC design
 - Many proven **compilers** for implementation on FPGA or on silicon (Synopsis, Xilinx, Mentor Graphics, Cadence, hp ...).
 - Verilog requires descriptions at a lower level. It is most commonly used to design signal processing circuits. It is the most used in America.

Hardware description in VHDL

- Description of a Digital System:

A digital system is described by its inputs and outputs, where the outputs depend on the inputs.



```
entity F is  
    port (A, B: in bit; Y out bit);  
end F
```

The designs are made up of Entity and Architecture.

- *Entity*
 - Defines externally a circuit or subcircuit
 - Name and number of ports, input and output data types
 - You have all the information you need to connect your circuit to other circuits.
- *Architecture*
 - Defines the internals of the circuit
 - Internal signals, functions, procedures, constants, etc.
- There can be several architectures for an entity

VHDL Basics

entity F is

port (A, B: in bit; Y out bit);
end F

Inputs and
outputs

architecture circuit of F is

signal D, E: bit_vector(1 downto 0);
signal H: bit;

Internal signals

begin

(...)

end architecture circuit;

functionality

Signals and variables

- Signals:
 - Represent memory elements or connections
 - The ports of an entity
 - They are defined in the architecture before BEGIN, which allows us to make connections between different modules.

signal <= value; The value is assigned after an infinitesimal delay
- Variables (**We are not going to use them**):
 - Only defined and used within processes (cannot be shared).
 - Can be used as indexes (loop instructions or component modeling)
 - Variables do NOT represent memory connections or states

variable := value; The value is assigned at present time

VHDL Basics

```
-- comment
```

```
ENTITY name IS
```

```
    GENERIC (parameter list);
```

```
    PORT (io list);
```

```
END name;
```

parameter list:

parameter_i : type_name; ...

delay: time; error: boolean

io list:

port_i : io_type type_name; ...

io_type

**IN | OUT | INOUT | BUFFER
| BUFFER**

*inputA: in bit_vector(7 downto 0);
outputC: out std_logic;*

Types of data

STD_LOGIC (one bit enriched):

- 'U': uninitialized. This signal hasn't been set yet.
- 'X': unknown. Impossible to determine this value/result.
- '0': logic 0
- '1': logic 1
- 'Z': High Impedance
- 'W': Weak signal, can't tell if it should be 0 or 1.
- 'L': Weak signal that should probably go to 0
- 'H': Weak signal that should probably go to 1
- '-': Don't care.

• **STD_LOGIC_VECTOR (range):**

- Vector of std_logics

BOOLEAN {TRUE,FALSE}

CHARACTER {ascii}

STRING {ascii}

INTEGER [range]

NATURAL [range].

POSITIVE [range].

REAL [range]

TIME

Two forms for range:

- n_min **TO** n_max
- n_max **DOWNTO** n_min

Assignment of values

- By default it works in binary
 - A single quotation mark is used for a bit: '0' or '1'.
 - Double quotation marks are used for multiple bits: "0011".
- Other formats can be used:
 - b "0011" --binary (No need to put *b* because it is the default case)
 - o "4567" 8#4567# --octal
 - x "9AB" 16#9AB# --hexadecimal
 - D "6789" 10#6789# --decimal

VHDL Basics

- Constants and new data types can be defined
- **Constants:** **constant constant** constant_name : type := value;
- Examples:
 - constant BUS_WIDTH : integer := 8;
 - constant FOUR_ONES : std_logic_vector(3 downto 0):= "1111";
 - constant PERIOD : time := 10 ns;
- **Types:** **type type** type_name is *definition*;
- Example with two dimensions:

```
type WORD_8 is STD_LOGIC_VECTOR (7 downto 0);
type TAB12 is array (11 downto 0) of WORD_8;
signal WORD_A : WORD8; signal TAB_A : TAB12;
WORD_A <= "00000001"; WORD_B <= "00000010";
TAB_A(11) <= WORD_A; TAB_B(0) <= WORD_B;
```

Types and constants
can be grouped into
packages for reuse

Functional description of an architecture

```
ARCHITECTURE arch_name OF entity_name IS
  signal_i;
BEGIN
  -- concurrent rulings
    AND, OR, NOT, NAND, XOR, +, - ...
    WAIT
  -- procedural (sequential ) assertions
  PROCESS (sensitivity list)
    variable _j;
  BEGIN
    -- assignments, conditional statements, loops
  END PROCESS;
END arch_name;
```

Functionality can be represented in two ways

- Processes:
 - A set of sequential statements.
 - The order of the statements can affect the result.
- Concurrent sentences:
 - Independent sentences and process invocations. They are executed in parallel.
 - The order they are written is irrelevant
- Processes are concurrent with each other
 - But execution is sequential within each process

VHDL Basics

Architecture ver1 of test is
begin

```
c <= a and b;  
z <= c when e = '1' else 'Z';  
seq: process (a, b)
```

Concurrent statements

```
begin
```

```
  if a = b then  
    f <= a or b;  
  end if;  
  If a = "1000" then  
    d <= f;  
  end if;
```

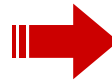
Sequential sentences:
Only within the processes

```
  end process;  
end ver1;
```

VHDL Basics

- Basic operators:

abs
**, /, mod, rem*
+ (sig.), - (sig)
+, -, &
and, or, nand, nor ...



```
y <= (x1 and x2) or d(0);  
y(1) <= x1 and not x2;
```

```
y: bit_vector (1 downto 0);  
y <= x1&x2;
```

VHDL Basics

- Concurrent statements (I):

```
signal_name <= value_1 when condition1 else  
               value_2 when condition2 else  
               ...  
               value_i when conditioni else  
               other_value;
```



```
output <= "00" when input = "0001" else  
          "01" when input = "0010" else  
          "10" when input = "0100" else  
          "11";
```

VHDL Basics

- Concurrent statements (II):

```
with identifier select  
signal_name <= value_1 when value_identifier1,  
                value_2 when identifier_value2,  
                ...  
                value_i when identifier_valuei, other_value when  
others;
```

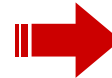


```
with select input  
output <= "00" when "0001",  
          "01" when "0010",  
          "10" when "0100",  
          "11" when others;
```

VHDL Basics

- Conditional sequential sentences (I):

```
if cond_1 then
...    --sequential statements
elsif cond-n then
...    --sequential statements
else ... --sequential statements
end if;
```



```
if a = b then
    c <= a or b;
elsif a < b then
    c <= b;
else
    c <=
    "0000";
end if;
```

VHDL Basics

- Conditional sequential sentences (II):

```
case expression is  
  when choice_1 => ... --seq. statements;  
  when choice_n => ... --seq. statements;  
  when others => ... --seq. statements;  
end case;
```



```
case RGB is  
  when "111" => r <= 0;  
  when "100" => r <= 1;  
  when "110" => r <= 1;  
  when others => r <= 0;  
end case;
```

Temporary model with concurrent sentences

-- Option 1:

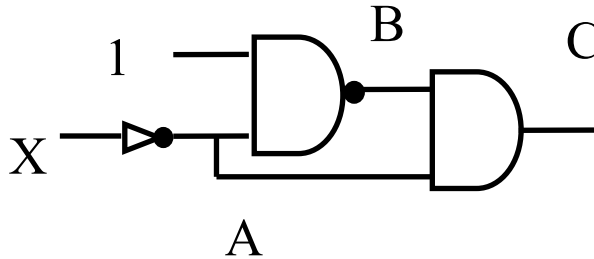
BEGIN

a <= NOT x;

b <= a NAND '1';

c <= a AND b;

END example ;



-- Option 2:

BEGIN

a <= NOT x;

c <= a AND b;

b <= a NAND '1';

END example ;

Initial values: X = '1' A = '0'; B = '1'; C = '0';

If **input X goes from '1' to '0'** what happens to A, B and C?

Case 1: the assignments are **immediate** and the code is simulated in the order in which it is written.

A: '0' => '1'

B: '1' => '0'

C: '0' => '0'

No good

The order would
matter!

A: '0' => '1'

C: '1' => '1'

B: '1' => '0'

Temporary model with concurrent sentences

-- Option 1:

BEGIN

a <= NOT x;

b <= a NAND '1';

c <= a AND b;

END example ;

Initial values: X = '1' A = '0'; B = '1'; C = '0';

X goes from '1' to '0' at T = 1ns;

Case 2: the assignments take an **infinitesimal delay Δ** and every time there is a change in an input, the lines that use it as input are simulated.

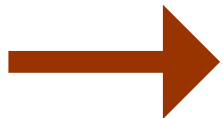
T = 1 ns;
A: '0' => '1' at T + Δ
B No change
C No change

The simulator
advances



T = 1 + Δ ;
A = '1'
B: '1' => '0' at T + 2* Δ
C : '0' => '1' in T + 2* Δ

The simulator
advances



T = 1 + 2* Δ ;
A = '1'; B = '0'; B = '0'; A
= '1'; B = '0'.
C : '1' => '0' in T + 3* Δ

The simulator
advances



T = 1 + 3* Δ ;
A = '1'; B = '0';
B = '0'; A = '1';
B = '0'.
C = '0'

Temporary model with concurrent sentences

```
-- Option 2:  
BEGIN  
  a <= NOT x;  
  c <= a AND b;  
  b <= a NAND '1';  
END example ;
```

Initial values: $X = '1'$ $A = '0'$; $B = '1'$; $C = '0'$;

X goes from '1' to '0' at $T = 1\text{ns}$;

Case 2: the assignments take an **infinitesimal delay Δ** and every time there is a change in an input, the lines that use it as input are simulated.

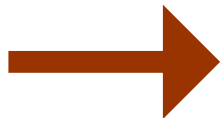
$T = 1\text{ ns};$
 $A: '0' \Rightarrow '1'$ at $T + \Delta$
B No change
C No change

The simulator
advances



$T = 1 + \Delta;$
 $A = '1'$
 $C: '0' \Rightarrow '1'$ in $T + 2*\Delta$
 $B: '1' \Rightarrow '0'$ at $T + 2*\Delta$

The simulator
advances



$T = 1 + 2*\Delta;$
 $A = '1'; B = '0'; B = '0'; A$
 $= '1'; B = '0'.$
 $C: '1' \Rightarrow '0'$ in $T + 3*\Delta$

The simulator
advances



$T = 1 + 3*\Delta;$
 $A = '1'; B = '0';$
 $B = '0'; A = '1';$
 $B = '0'.$
 $C = '0'$

Process simulation

- My_process: process (sensitivity list)
- A process is simulated if any signal in its sensitivity list changes
- If a process represents combinational logic, its list should include all the inputs of each line.
- If a process represents sequential logic (a register or a counter), its list should only include the clock signal and the reset or clear in case it is asynchronous.

Temporal model of a process

Example: process (x)

BEGIN

a <= NOT x;

b <= a NAND '1';

c <= a AND b;

END process;

Initial values: X = '1' A = '0'; B = '1'; C = '0';

X goes from '1' to '0' at T = 1ns;

When a process is executed, its lines are executed sequentially one at a time.

T = 1 ns;

A: '0' => '1' in T + Δ

B No change

C No change

The simulator
advances



T = 1 + Δ ;

A = '1'

As X does not change, the process is not rerun.

As X in the sensitivity list has changed, the following process is executed

Temporal model of a process

Example: process (x,a,b) Initial values: X = '1' A = '0'; B = '1'; C = '0';

BEGIN

a <= NOT x;

b <= a NAND '1';

c <= a AND b;

END example ;

X goes from '1' to '0' at T = 1ns;

When a process is executed, its lines are executed sequentially one at a time.

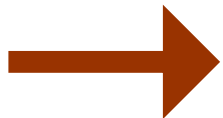
T = 1 ns;
A: '0' => '1' in T + Δ
B No change
C No change

The simulator
advances



T = 1 + Δ ;
A = '1'
B: '1' => '0' at T + 2* Δ
C : '0' => '1' in T + 2* Δ

The simulator
advances



T = 1 + 2* Δ ;
A = '1'; B = '0'; B = '0'; A
= '1'; B = '0'.
C : '0' => '1' in T + 3* Δ

The simulator
advances



T = 1 + 3* Δ ;
A = '1'; B = '0';
B = '0'; A = '1';
B = '0'.
C = '1'

Temporal model of a process

Example: process (x)

BEGIN

a <= '1';

IF (X = '0') then

a <= '0';

END IF;

END process;

Initial values: X = '1' A = '1';

X goes from '1' to '0' at T = 1ns;

When a process is executed, its lines are executed sequentially one at a time.

T = 1 ns;

A: '1' => '1' in 1ns + Δ

A: '0' => '1' in 1ns + Δ

T = 1 + Δ ;

A = '0'

The process does not run again until X changes.

The second assignment
overwrites the first

Implementation of combinational processes

Ex_combinational: Process (a, b, c)

Begin

z <= '0';

IF (B='1') AND (C='1') THEN

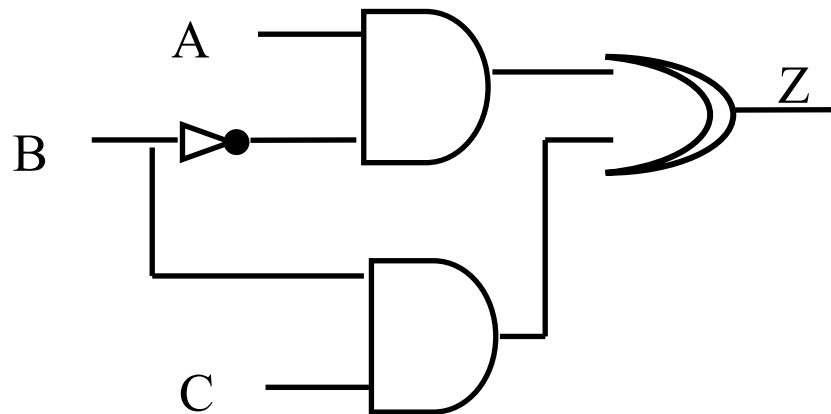
z <= '1';

ELSIF (A = '1') AND (B='0') THEN

z <= '1';

END IF;

End process;



The output must be defined for any input!

Ex_combinational: Process (a, b, c)

Begin

IF (B='1') AND (C='1') THEN

z <= '1';

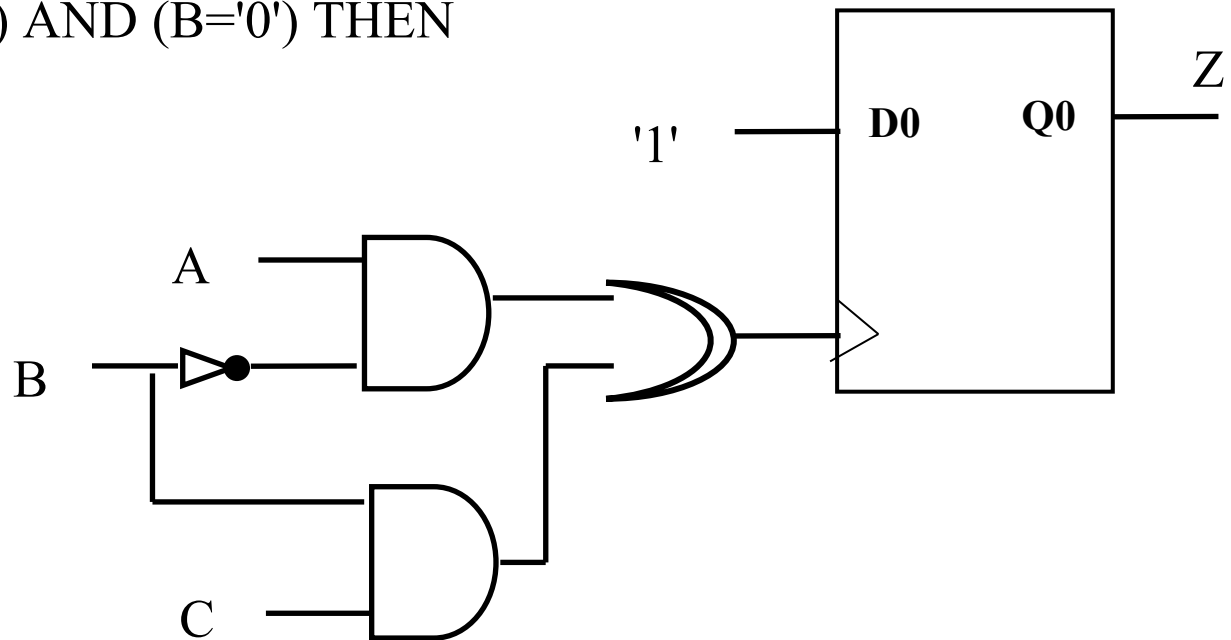
ELSIF (A = '1') AND (B='0') THEN

z <= '1';

END IF;

End process;

An unwanted latch appears for storing Z when the condition is not met



A signal must not be assigned in several processes

Example1: process (x, z)

BEGIN

a <= '0';

IF (X = '1') AND (Z = '1') then

a <= '1';

END IF;

END process;

Example2: process (y, z)

BEGIN

a <= '0';

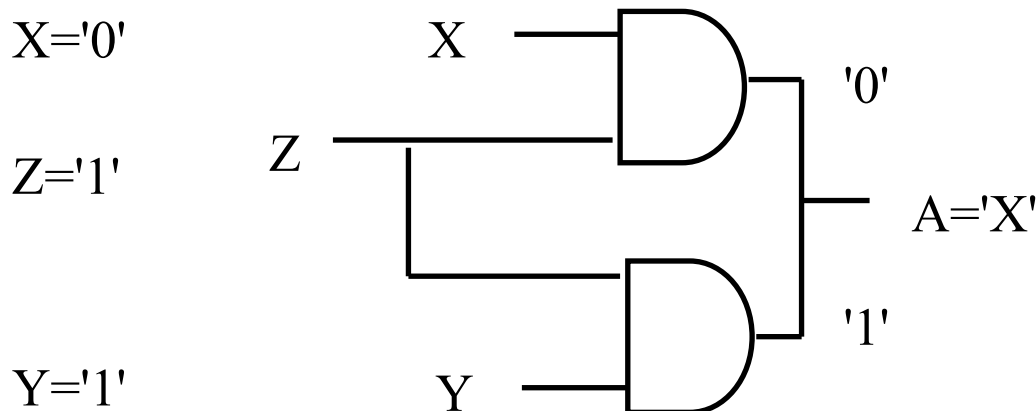
IF (Y = '1') AND (Z = '1') then

a <= '1';

END IF;

END process;

This is a short circuit: *multiple drivers!*



A signal must not be assigned in several processes

Example1: process (x, z)

BEGIN

a <= '1';

IF (X = '1') AND (Z= '1') then

a <= '0';

END IF;

END process;

Example2: process (y, z)

BEGIN

a <= '1';

IF (Y = '1') AND (Z= '1') then

a <= '0';

END IF;

END process;

Solution:

Example1: process (x,y)

BEGIN

a <= '1';

IF ((X = '1') AND (Z= '1')) or ((Y = '1') AND (Z= '1'))
then

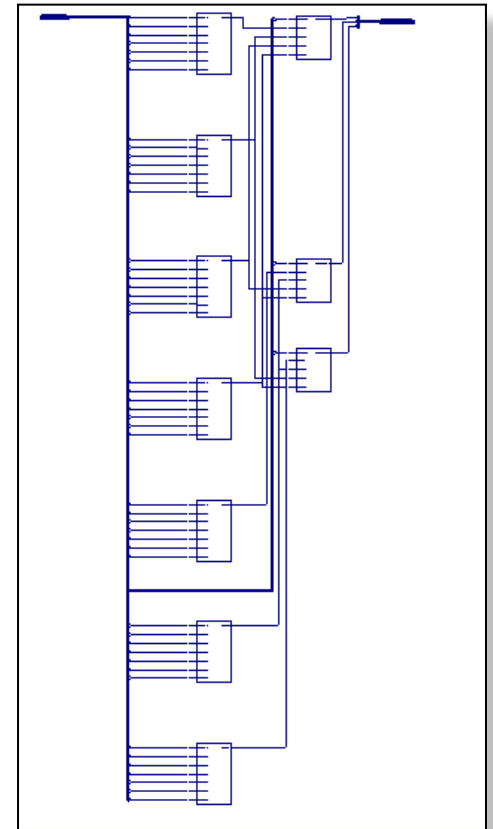
a <= '0';

END IF;

END process;

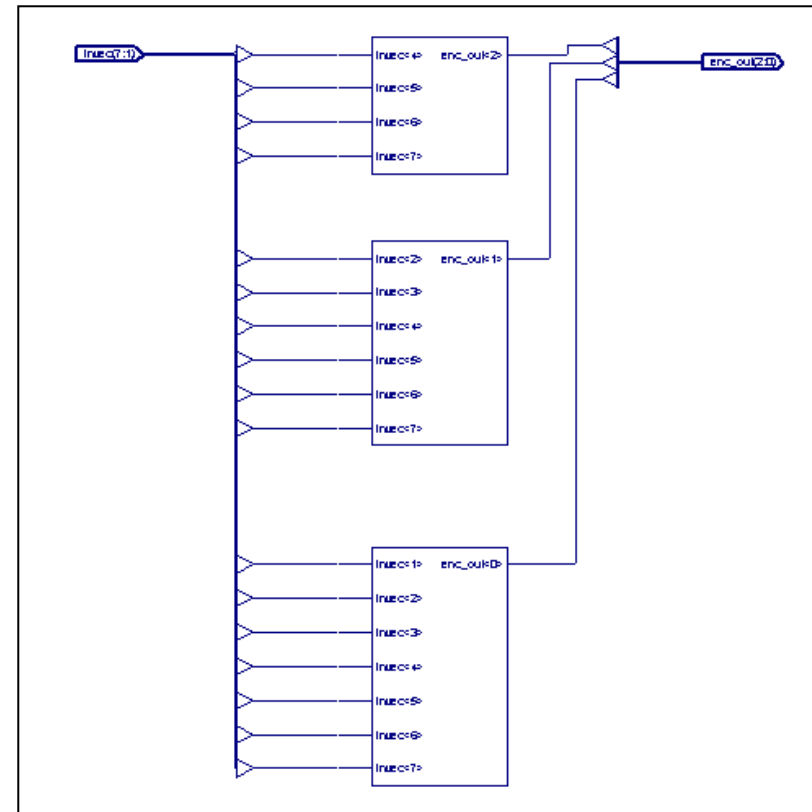
VHDL Basics

```
library ieee;
use ieee.std_logic_1164.all;
entity encoder is
    port (invec: in std_logic_vector(7 downto 0); enc_out: out std_logic_vector(2 downto 0));
end encoder;
architecture rtl of encoder is
begin
    encode: process (invec)
    begin
        case invec is
            when "00000001" => enc_out <= "000";
            when "00000010" => enc_out <= "001";
            when "00000100" => enc_out <= "010";
            when "00001000" => enc_out <= "011";
            when "00010000" => enc_out <= "100";
            when "00100000" => enc_out <= "101";
            when "01000000" => enc_out <= "110";
            when "10000000" => enc_out <= "111";
            when others => enc_out <= "000";
        end case;
    end process;
end rtl;
```



VHDL Basics

```
library ieee;
use ieee.std_logic_1164.all;
entity encoder is
  port (invec:in std_logic_vector(7 downto 0); enc_out:out std_logic_vector(2 downto 0));
end encoder;
architecture rtl of encoder is
begin
  process (invec)
  begin
    if invec(7) = '1' then enc_out <= "111";
    elsif invec(6) = '1' then enc_out <= "110";
    elsif invec(5) = '1' then enc_out <= "101";
    elsif invec(4) = '1' then enc_out <= "100";
    elsif invec(3) = '1' then enc_out <= "011";
    elsif invec(2) = '1' then enc_out <= "010";
    elsif invec(1) = '1' then enc_out <= "001";
    elsif invec(0) = '1' then enc_out <= "000";
    else enc_out <= "000";
    end if;
  end process;
end rtl;
```



VHDL Basics

```
library ieee;
use ieee.std_logic_1164.all;
entity encoder is
port (invec: in std_logic_vector(7 downto 0); enc_out: out std_logic_vector(2 downto 0));
end encoder;
architecture rtl of encoder is
begin
    enc_out <= "111" when invec(7) = '1' else
               "110" when invec(6) = '1' else
               "101" when invec(5) = '1' else
               "100" when invec(4) = '1' else
               "011" when invec(3) = '1' else.
               "010" when invec(2) = '1' else
               "001" when invec(1) = '1' else.
               "000" when invec(0) = '1' else
               "000";
end rtl;
```

Synthesis of Sequential Sentences

- Example: 32 bit register

Reg: process (clk)

begin

if (clk'event and clk = '1') then

if (reset = '1') then

Dout <= x "00000000";

else

if (load='1') then

Dout <= Din;

end if;

end if;

end if;

end process;

Example: 8-bit counter

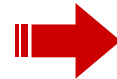
```
use IEEE.std_logic_arith.all; --libraries for arithmetic operations
use IEEE.std_logic_unsigned.all;

...
process (clk) -- counter with load signal and count_enable
begin
    if clk='1' and clk'event then
        if reset='1' then
            count <= (others => '0'); -- sets all to 0
        elsif load = '1' then -- if load is 1 the counter loads the input data
            count <= D_in;
        elsif count_enable='1' then
            count <= count + "00000001"; --the counter counts.
        end if;
    end if; end process;
```

VHDL Basics

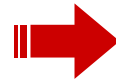
- Loops (only within processes):

```
for loop_var in range loop  
...  --sequential statements  
end loop;
```



```
for i in 3 downto 0 loop  
-- i is a variable that does not  
need  
-- to be declared  
    d(i) <= a(i+1);  
end loop;
```

```
while loop condition  
...  --sequential statements  
end loop;
```



```
while counter > 0 loop  
counter := counter-1;  
result := result+data;  
end loop;
```

We will not use them

VHDL Basics

- Sequential expressions:

```
wait;  
wait on signal_list;  
wait for time_expression;  
wait until condition;
```

To wait 10ns in a simulation
wait for 10ns;

```
signal'event  
signal'last_event  
signal'last_value
```

For a register to do something
only on the rising edge:

if clock'event and clock = '1';

VHDL Basics

- Structural description: (*structural*)
 - Describes the interconnections between different modules
 - These modules can in turn have a structural or behavioral model.
 - Normally this is a higher level description

VHDL Basics

- Structural Description:

```
ARCHITECTURE arch_name OF entity_name IS
  COMPONENT component_name
    GENERIC (...);
    PORT (...);
  END COMPONENT;
signal declarations;
BEGIN
  component_i: component_name
    GENERIC MAP (parameter_value)
    PORT MAP (io_name)
END arch_name;
```

Example of structural description

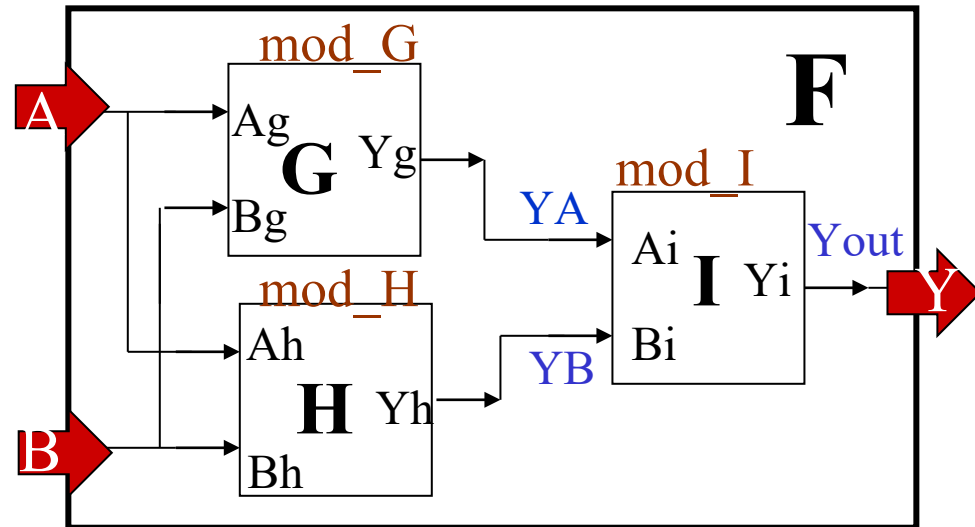
architecture structure of F is

```
component G
  port (Ag, Bg: in bit; Yg: out bit);
end component;
component H
  port (Ah, Bh: in bit; Yh: out bit);
end component;
component I
  port (Ai, Bi: in bit; Yi: out bit);
end component;
signal YA, YB, Yout: bit;
```

begin

```
  mod_G: G port map (Ag=>A; Bg=>B; Yg=>YA);
  mod_H: H port map (Ah=>A; Bh=>B; Yh=>YB);
  mod_I : I port map (Ai=>YA; Bi=>YB; Yi=>Yout);
  Y<=Yout;
```

end structure;

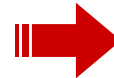


VHDL Basics

- **GENERATE:** Component generation sequences allow the creation of one or more copies of a set of interconnections, which facilitates the design of circuits by means of structural descriptions.

```
for index in range generate  
  -- concurrent instructions  
end generate;
```

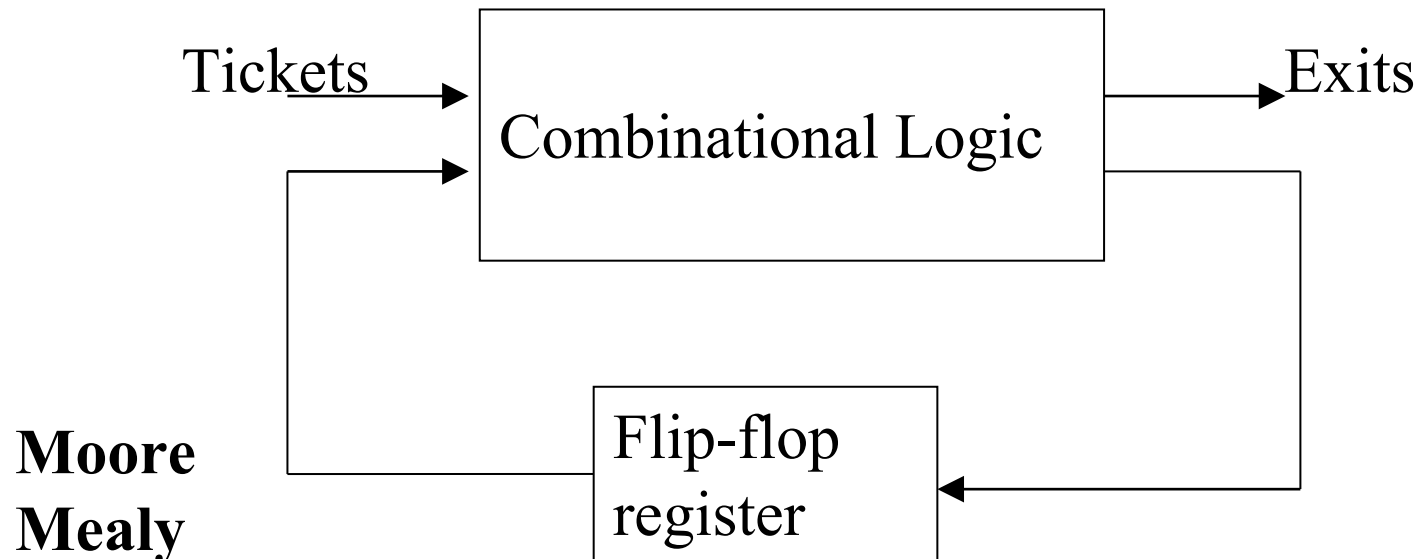
We will not use it



```
component comp  
  port( x: in bit; y: out bit);  
end comp  
...  
signal a, b: bit_vector(0 to 7)  
...  
gen: for i in 0 to 7 generate  
  u: comp port map (a(i),b(i));  
end generate gen;
```

Finite State Machine (FSM)

- Every sequential circuit is divided into a combinational circuit that implements the output of the circuit and the transition to the next state and some storage elements.



Finite State Machine (FSM)

architecture behavior of *FSM* is

internal signal declarations;

begin

synchronous: **process** (*clock, reset*)

begin

vhdl indicating flip-flops

synchronous **end process**;

comb: **process** (*sensitivity list*)

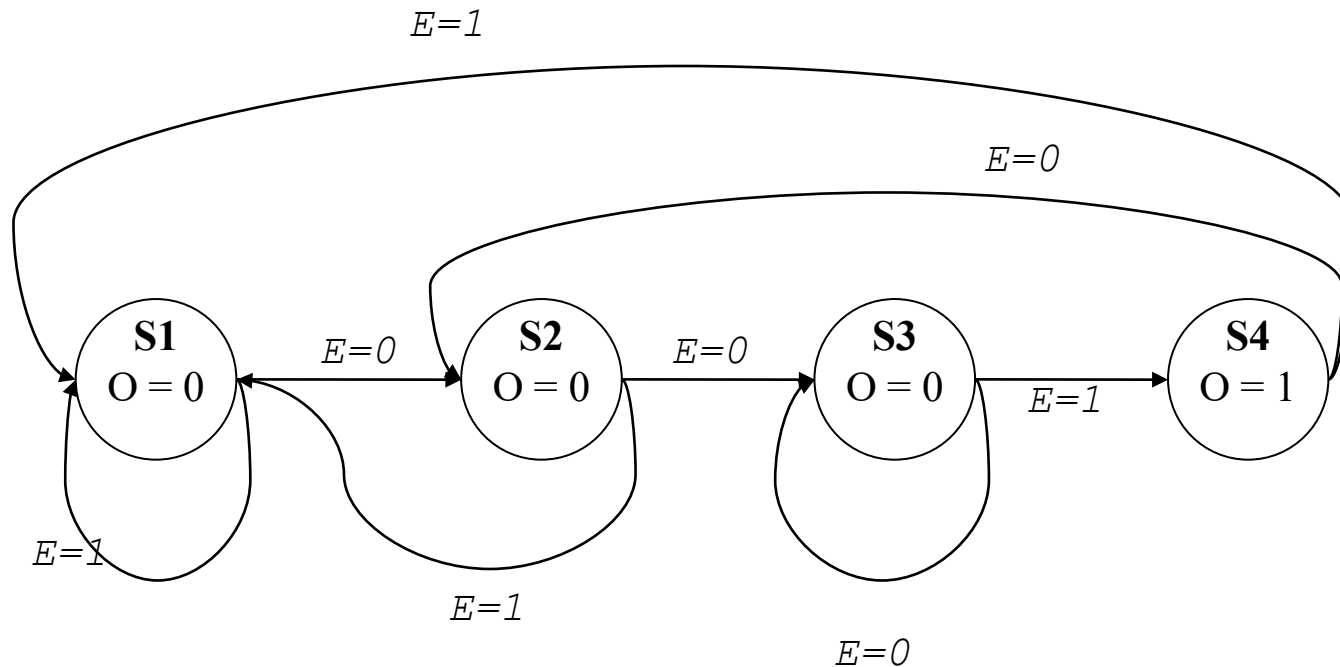
begin

vhdl indicating combinational logic

end process comb;

end behavior;

Finite State Machine (FSM)



Finite State Machine (FSM)

```
IEEE library;  
use IEEE.std_logic_1164.all;  
entity FSM is  
    port(reset, E, clk: in bit; O: out bit);  
end FSM;  
architecture ARCH of FSM is  
    type STATES is (S1, S2, S3,S4);  
    signal STATE, SIG_STATE: STATES;
```

```
begin
```

```
    SYNCHRONOUS: process(clk,reset)  
    begin  
        if reset='1' then  
            STATUS<=S1;  
        elsif clk'event and clk='1' then  
            STATE<= SIG_STATE;  
        end if;  
    end process SYNCHRONO;
```

```
end ARCH;
```

```
    COMB: process(STATUS,E)  
    begin  
        case STATE is  
            when S1 =>  
                O <= '0';  
                if (E='0') then SIG_STATE<=S2;  
                else STATUS_SIG<=S1;  
                end if;  
            when S2 =>  
                O <= '0';  
                if (E='0') then SIG_STATE<=S3;  
                else STATUS_SIG<=S1;  
                end if;  
            when S3 =>  
                O <= '0';  
                if (E='0') then SIG_STATE<=S3;  
                else SIG_STATE<=S4;  
                end if;  
            when S4 =>  
                O <= '1';  
                if (E='0') then SIG_STATE<=S2;  
                else STATUS_SIG<=S1;  
                end if;  
        end case;  
    end process control;
```